VLSI PDA HW2

314551042 交大資科工碩一 王邦碩

I. The final cut size and the runtime of each testcase. Paste the screenshot of the result of running the HW2_grading.sh as the picture shown below.

```
+-----------------------------------------------+
|                                               |
|    This script is used for PDA HW2 grading.   |
|                                               |
+-----------------------------------------------+
host name: ic21
compiler version: g++ (GCC) 7.3.0

grading on 314551042:
 checking item          | status
------------------------|--------
 correct tar.gz         | yes
 correct file structure | yes
 have README            | yes
 have Makefile          | yes
 correct make clean     | yes
 correct make           | yes

  testcase |      #ways |   cut size |    runtime | status
-----------|------------|------------|------------|--------
   public1 |          2 |        587 |       0.20 | success
   public1 |          4 |       1090 |       0.41 | success
   public2 |          2 |      15523 |      30.58 | success
   public2 |          4 |      21301 |      43.30 | success
+-----------------------------------------------+
|                                               |
|   Successfully write grades to HW2_grade.csv  |
|                                               |
+-----------------------------------------------+
```

II. The details of your implementation containing explanations of the following questions.

A. Where is the difference between your algorithm and FM Algorithm described in class?

The most of the algorithm flow is the same as FM Algorithm described in class. The small difference is that I set tolerance constant for early termination. The algorithm stops if the current cut size remains worse than the best cut size for <negTol> consecutive iterations. In the BucketList::getMax(), I also set a tolerance value to pick cell from the unbalanced partition set for only few chance. Besides, I use BFS to get the initial partition.

B. Did you implement the bucket list data structure?

I implement an array data structure to store all possible 2pMax + 1 buckets, every bucket also be implemented as list structure to store the cell id with the corresponding gain. With getMax(), getGain(), deleteNode() function, we can perform FM algorithm with a lower complexity.

```
class GainBucket {
private:
    int numNodes;
    int pMax;
    boost::container::vector<std::list<int>> gainBuckets;
    boost::container::vector<int> gainTable;
    boost::container::vector<std::list<int>::iterator> nodePtr;
    int countTol;
public:
    GainBucket(){}
    GainBucket(int _numNodes, int _pMax) : numNodes(_numNodes),
    void initBucket(boost::container::vector<int> gain){ ···
    int getGain(int cID){return gainTable[cID];}
    int getMax(boost::container::vector<int> nodePartition, boos

    void deleteNode(int cID){ ···
    void adjust(int cID, int val){ ···
    void dump(){ ···
    void dumpSize(){ ···
};
```

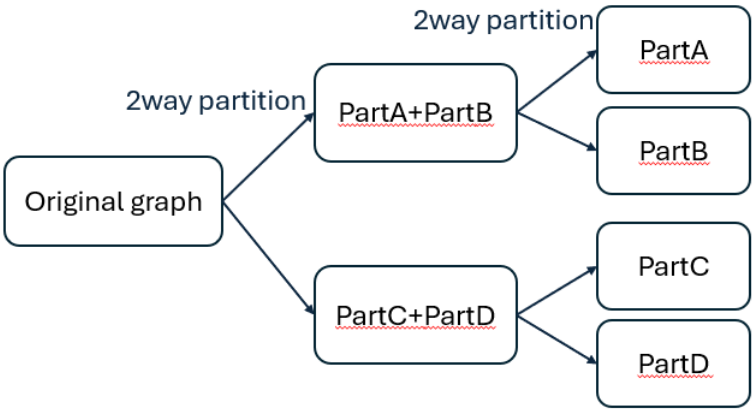
```
Bucket index i(97 ~ -97): num of cell in bucket[i]
8: 2
6: 5
5: 9
4: 36
3: 266
2: 1527
1: 6002
0: 15095
-1: 40137
-2: 39460
-3: 32492
-4: 35858
-5: 12278
-6: 15715
-7: 2654
-8: 1633
-9: 7338
-10: 101
-20: 1
-39: 1
-59: 1
-93: 1
-97: 1
```

C. How did you find the maximum partial sum and restore the result?

Everytime after switching a cell to another partition, compare the current cutsize to the history best partition(min cutsize) and replace it if better. After all iteration is completed, restore the partition configuration to the best one which is recorded.

D. How did you modify the FM algorithm to handle four-way min-cut partitioning? You could use flow chart(s) and/or pseudo code to help elaborate your algorithm.

I employed a recursive approach using the two-way FM algorithm. The first partition separates the graph to 2 part with each sumSize between 45%~55%. Then independently apply the same two-way FM algorithm to each of the subgraphs to get 4 partition set.



III. What have you learned from this homework? What problem(s) have you encountered in this homework?

In the proces of dealing with HW2, the most time-consuming part is to lower the complexity to speed up the program. My initial implementation failed to complete within the required runtime. Therefore I spend most of my time to design and modify the data structure, especiailly bucket list, like how to efficiently store and index the gain of particular cell.